BTS CIEL · PROGRAMMATION C++ · EXERCICES

Exercices — Chapitre 3 : Conditions et logique

Cours associé : `C03_Conditions_et_logique` · **Fichiers de départ :** `02_Exercices/fichiers/E03/`

> **Méthode :** créez un dossier `E03/` et un fichier par exercice. Pour chaque condition, testez **les valeurs limites** : la borne elle-même et la valeur juste à côté.

N°	Titre	Niveau	Notions du cours
3.1	Contrôle d'une mesure	1 — Application	§1, §2
3.2	Pair ou impair, maximum	2 — Entraînement	§1, §2
3.3	Menu de diagnostic	2 — Entraînement	§3
3.4	Trouver l'erreur	3 — Problème	§1, §2
3.5	État d'une batterie 12 V	4 — Défi	§1 à §4

Niveau 1 — Application directe

Exercice 3.1 — Contrôle d'une mesure

Objectif : écrire une suite de conditions `if / else if / else`.

Énoncé : demandez une température entière et affichez :

- `Temperature basse` si elle est inférieure à 0 ;
- `Temperature normale` si elle est comprise entre 0 et 30 inclus ;
- `Temperature elevee` si elle est supérieure à 30.

Contraintes : une seule structure `if / else if / else` ; accolades obligatoires.

Tests :

Saisie	Affichage attendu
`-1`	`Temperature basse`
`0`	`Temperature normale`
`15`	`Temperature normale`
`30`	`Temperature normale`
`31`	`Temperature elevee`

Niveau 2 — Entraînement

Exercice 3.2 — Pair ou impair, maximum

Objectif : combiner `%` et des comparaisons ; traiter tous les cas, y compris l'égalité.

Énoncé : demandez deux entiers.

1. Indiquez si le premier est pair ou impair.
2. Affichez le plus grand des deux, ou un message spécifique s'ils sont égaux.

Tests :

Saisies	Affichage attendu
`4` `9`	`4 est pair` puis `Le plus grand est 9`
`7` `2`	`7 est impair` puis `Le plus grand est 7`
`5` `5`	`5 est impair` puis `Les deux nombres sont egaux`
`-3` `-8`	`-3 est impair` puis `Le plus grand est -3`

> **Attention :** en C++, `-3 % 2` vaut `-1`, et non `1`. Votre test de parité doit fonctionner aussi pour les nombres négatifs.

Exercice 3.3 — Menu de diagnostic

Objectif : utiliser `switch` avec `break` et `default`.

Énoncé : affichez le menu ci-dessous, demandez un choix, puis affichez le message correspondant avec `switch`.

```
1. Afficher l'etat
2. Demarrer
3. Arreter
4. Quitter
```

Contraintes : un `break` par `case` ; un `default` pour les choix non prévus.

Tests :

Choix	Affichage attendu
`1`	`Etat : systeme operationnel`
`2`	`Demarrage en cours`
`3`	`Arret en cours`
`4`	`Au revoir`
`7`	`Choix invalide`

Niveau 3 — Résolution de problèmes

Exercice 3.4 — Trouver l'erreur

Objectif : analyser une condition logique erronée et la corriger.

Énoncé : un technicien veut afficher `Alarme` lorsque la tension sort de la plage 11 V à 14 V incluse, et `Tension correcte` sinon. Son programme (`ex3_4.cpp`) affiche `Alarme` pour 12 V.

```cpp
#include <iostream>

int main() {
    double tension{12.0};

    if (tension < 11.0 && tension > 14.0) {
        std::cout << "Tension correcte\n";
    } else {
        std::cout << "Alarme\n";
    }

    return 0;
}
```

1. Pourquoi la condition est-elle toujours fausse ?
2. Corrigez le programme de deux façons :
   - a) en gardant une condition qui décrit une tension **incorrecte** : changez l'opérateur logique, puis vérifiez quel message doit être associé à chaque branche ;
   - b) en réécrivant la condition pour qu'elle décrive une tension **correcte**.
3. Pour chaque version, testez les valeurs du tableau.

Tests :

Tension	Affichage attendu
`10.9`	`Alarme`
`11.0`	`Tension correcte`
`12.0`	`Tension correcte`
`14.0`	`Tension correcte`
`14.1`	`Alarme`

Niveau 4 — Défi

Exercice 3.5 — État d'une batterie 12 V

Objectif : contrôler une saisie, classer une valeur dans des plages contiguës, renvoyer un code de retour significatif.

Contexte : un système de supervision surveille la batterie de secours d'une baie réseau. La tension mesurée indique l'état de la batterie.

Énoncé : demandez la tension mesurée (nombre décimal), puis :

1. si la saisie n'est pas numérique : message d'erreur sur `std::cerr`, code de retour **1** ;
2. si la tension n'est pas comprise entre 0 et 20 V inclus (valeur physiquement impossible pour ce capteur) : message d'erreur sur `std::cerr`, code de retour **1** ;
3. sinon, affichez l'état selon le tableau ci-dessous et renvoyez le code indiqué.

Tension (V)	État affiché	Code de retour
de 0 à moins de 11,8	`DECHARGEE`	2 (alarme)
de 11,8 à moins de 12,4	`PARTIELLE`	0
de 12,4 à 13,0 inclus	`CHARGEE`	0
plus de 13,0 à 14,4 inclus	`EN CHARGE`	0
plus de 14,4 à 20 inclus	`SURTENSION`	2 (alarme)

Contraintes :

- les seuils sont des constantes `constexpr` ;
- une seule structure `if / else if / else` pour la classification, sans tester deux fois la même borne ;
- affichage au format `Etat : CHARGEE` ;
- vérifiez le code de retour avec `echo $?` (ou `$LASTEXITCODE` sous PowerShell).

Tests :

Saisie	Affichage attendu	Code
`abc`	message d'erreur	1
`-1`	message d'erreur	1
`25`	message d'erreur	1
`11.79`	`Etat : DECHARGEE`	2
`11.8`	`Etat : PARTIELLE`	0
`12.39`	`Etat : PARTIELLE`	0
`12.4`	`Etat : CHARGEE`	0
`13.0`	`Etat : CHARGEE`	0
`13.01`	`Etat : EN CHARGE`	0
`14.4`	`Etat : EN CHARGE`	0
`14.41`	`Etat : SURTENSION`	2